

JavaScript Metaprogramming

[Back to JS page](#)

Table of Contents

- [JavaScript Metaprogramming](#)
 - [Inspecting Objects](#)
 - [Example 1](#)
 - [Modify Objects](#)
 - [Example 2](#)
 - [Generate Dynamic Code](#)
 - [Example 3](#)
 - [Proxy Metaprogramming](#)
 - [Example 4](#)
 - [Proxy with Reflect](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

Metaprogramming

Metaprogramming is a programming technique where programs can treat other programs as their data. In JavaScript, metaprogramming allows you to:

- **Inspect** objects (get properties, check types)
- **Modify** objects (add/change properties)
- **Generate** dynamic code at runtime
- **Intercept** operations using Proxy and Reflect

Inspecting Objects

You can inspect objects using methods like `Object.keys()` , `Object.getOwnPropertyNames()` , and `typeof` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Metaprogramming</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Inspecting Objects</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

let text = "Properties:<br>";
const keys = Object.keys(person);
for (let key of keys) {
  text += key + ": " + person[key] + " (" + typeof person[key] + ")<br>";
}
text += "<br>Number of properties: " + keys.length;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Modify Objects

You can modify objects at runtime by adding, changing, or deleting properties:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Metaprogramming</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Modify Objects</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe"
};

// Add a new property
Object.defineProperty(person, "age", {
  value: 30,
  writable: true,
  enumerable: true
});

// Modify an existing property
Object.defineProperty(person, "firstName", {
  value: "Jane"
});

let text = "Modified object:<br>";
const keys = Object.keys(person);
for (let key of keys) {
  text += key + ": " + person[key] + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Generate Dynamic Code

The `Function()` constructor can create functions dynamically from strings:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Metaprogramming</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Generate Dynamic Code</h4>
<p id="demo"></p>

<script>
// Create a function dynamically using the Function constructor
const operation = "multiply";
let dynamicFunction;

if (operation === "add") {
  dynamicFunction = new Function("a", "b", "return a + b");
} else if (operation === "multiply") {
  dynamicFunction = new Function("a", "b", "return a * b");
} else if (operation === "power") {
  dynamicFunction = new Function("a", "b", "return Math.pow(a, b)");
}

document.getElementById("demo").innerHTML =
  "Dynamic function created for: " + operation + "<br>" +
  "5 * 3 = " + dynamicFunction(5, 3);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Proxy Metaprogramming

A `Proxy` object wraps another object and intercepts operations like property access, assignment, and deletion:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Metaprogramming</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Proxy Metaprogramming</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

const handler = {
  get: function(target, property) {
    if (property === "fullName") {
      return target.firstName + " " + target.lastName;
    }
    return target[property];
  },
  set: function(target, property, value) {
    if (property === "age" && (value < 0 || value > 120)) {
      console.log("Invalid age: " + value);
      return false;
    }
    target[property] = value;
    return true;
  }
};

const proxy = new Proxy(person, handler);

// Use the proxy
proxy.age = 25; // Valid
proxy.age = 200; // Invalid - will be rejected

document.getElementById("demo").innerHTML =
  "firstName: " + proxy.firstName + "<br>" +
  "fullName: " + proxy.fullName + "<br>" +
  "age: " + proxy.age;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Proxy with Reflect

The `Reflect` object provides methods for interceptable JavaScript operations. It's often used with Proxy:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Metaprogramming</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Proxy with Reflect</h4>
<p id="demo"></p>

<script>
const logger = {
  get: function(target, property) {
    console.log("Accessing property: " + property);
    return Reflect.get(target, property);
  },
  set: function(target, property, value) {
    console.log("Setting property: " + property + " = " + value);
    return Reflect.set(target, property, value);
  }
};

const user = {
  name: "Alice",
  role: "admin"
};

const proxy = new Proxy(user, logger);

// Operations are logged via the proxy
proxy.name = "Bob";
let role = proxy.role;

document.getElementById("demo").innerHTML =
  "name: " + proxy.name + "<br>" +
  "role: " + proxy.role + "<br>" +
  "(Check the browser console for logged operations)";
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Metaprogramming](#)